

CS100 HW8 游戏攻略

僵尸公敌 I, Zombie

注意事项

- **请为本次作业分配充足的时间（参考：至少 3 天），并且不要到临近截止日期时才开始！**
- 无论你是否准备立刻开工，[请先了解 CMake 的使用方式](#)，或阅读附录中的“首次运行”部分。Windows 下可以使用集成了生成工具的 Visual Studio，也可以使用 VS Code 配合 CMake Tools。macOS 下推荐安装 CMake 后使用命令行编译，并使用 VS Code 配合 CMake Tools；如果你已经熟悉 Xcode，也可以自行使用 Xcode 生成项目。请立刻下载所需的依赖，并且确保你能够正常编译并运行提供的代码框架。若遇到问题，尽早在 Piazza 上提问。
- 如果对任何部分有疑问，在你提问之前，请先在本说明或 FAQ 中寻找答案，或试着游玩样例游戏并观察样例游戏的处理方式。
- [请尽量在 Piazza 上公开提问而非私下询问 TA](#)。有相同疑问的其他同学也可以看到你的问题，你也能看到我们对更多问题的回答。这会更高效地解决你的疑问。
- **请一定经常保存备份（无论离线或在线）！**如果你为了添加某一点功能使得整个程序崩溃或是在修复时越修越糟，简单地回退一版并比较不同，就可以快速解决问题。

目录

1	好的, 所以这么多东西我怎么写?	4
2	GameWorld	4
2.1	GameWorld::Init()	4
2.2	GameWorld::Update()	6
2.3	GameWorld::CleanUp()	7
3	背景 (Background)	7
3.1	当被创建时	7
3.2	当 Update() 时	7
3.3	当被点击时	7
4	阳光数 (Sun Counter)	7
4.1	当关卡开始时	7
4.2	当 Update() 时	8
4.3	当被点击时	8
5	脑子 (Brain)	8
5.1	当被创建时	8
5.2	当 Update() 时	8
5.3	当被点击时	9
5.4	当与僵尸碰撞时	9
6	推进红线与关卡进度 (Deployment Line and Progress Meter)	9
6.1	当被创建时	9
6.2	当阶段完成时	9
6.3	当新阶段开始时	10
7	僵尸投放位 (Zombie Deployment Lane)	10
7.1	当被创建时	10
7.2	当 Update() 时	10
7.3	当被点击时	10
8	普通僵尸卡牌 (Regular Zombie Card)	11

8.1	当被创建时	11
8.2	当 Update() 时	11
8.3	当被点击时	11
9	冷却遮罩 (Cooldown Mask)	12
9.1	当被创建时	12
9.2	当 Update() 时	12
9.3	当被点击时	12
10	向日葵 (Sunflower)	12
10.1	当被创建时	12
10.2	当 Update() 时	12
10.3	当被点击时	13
10.4	当被僵尸吃掉时	13
11	豌豆射手 (Peashooter)	13
11.1	当被创建时	13
11.2	当 Update() 时	13
11.3	当被点击时	13
12	双发射手 (Repeater)	14
12.1	当被创建时	14
12.2	当 Update() 时	14
12.3	当被点击时	14
13	豌豆 (Pea)	14
13.1	当被创建时	14
13.2	当 Update() 时	15
13.3	当被点击时	15
13.4	当与僵尸碰撞时	15
14	坚果墙 (Wall-nut)	15
14.1	当被创建时	15
14.2	当 Update() 时	15
14.3	当被点击时	15

15 普通僵尸 (Regular Zombie)	16
15.1 当被创建时	16
15.2 当 Update() 时:	16
15.3 当被点击时	16
15.4 当发生碰撞时	16
16 路障僵尸 (Conehead Zombie)	17
16.1 当被创建时	17
16.2 当 Update() 时:	17
16.3 当发生碰撞时	17
17 铁桶僵尸 (Buckethead Zombie)	17
17.1 当被创建时	17
17.2 当 Update() 时:	18
17.3 当发生碰撞时	18
18 撑杆跳僵尸 (Pole Vaulting Zombie)	18
18.1 当被创建时	18
18.2 当 Update() 时:	18
18.3 当发生碰撞时	19
19 蹦极僵尸 (Bungee Zombie)	19
19.1 当被创建时	20
19.2 当 Update() 时	20
19.3 当发生碰撞时	20
20 附录 - 碰撞处理	20

1 好的，所以这么多东西我怎么写？

可想而知，从零开始写出一个完整的游戏绝不是能一蹴而就的小工程。本游戏的大量内容需要逐步完成。因此，我们将**推荐先实现的部分**以蓝色标出。我们建议你先阅读完整的题目说明，对这个游戏基本了解，之后再开工。写码时建议从**蓝色字体部分**开始逐步进行。

在提供的文件之中，**你不需要也不应当修改** `src/Framework` 路径下的文件。对于 `utils.hpp`，在图片素材加载错误时你可能需要修改其中 `ASSET_DIR` 字符串，并且你可以在其中定义新的项目来添加属于你自己的内容（详见 FAQ：怎么添加新内容）。

下面是对游戏中每个组成部分的介绍。其中以符号开头的列表项是无关顺序的，数字或字母开头的列表项表示需要按顺序执行。为了简化大家调试参数的部分，我们对于游戏机制还有贴图坐标等等数据都在具体对象中给出，参考具体数值你将会得到一个不错的画面效果。

但请注意，本文章也并不是样例游戏的 C++ 代码翻译为中文而已，只是给你提供实现游戏的思路，当你有了要实现的功能之后，仍然还有许多问题需要给你来思考，例如：

有很多不同种类植物、僵尸、僵尸卡牌，对于不同僵尸，它们有哪些相同，哪些不同？能否通过设定出一个基类使得接下来的都继承自同一基类？

点击僵尸卡牌.....点击投放位生成僵尸.....这些动作的完成需要依赖于上一次点击的状态，如何让程序记住这些状态该怎么实现？当然有很多种实现方式，尝试“隐形的兔子”也是一种选择。

2 GameWorld

2.1 GameWorld::Init()

你的 **GameWorld** 的 `Init()` 函数应当：

- **初始化任何用于记录关卡数据的成员变量**。例如，初始阳光为 150，每一行的脑子均未被吃掉，玩家当前没有选中任何僵尸卡牌，当前推进阶段为第 1 阶段。
- **为需要显示的关卡数据创建文本显示**。需要使用 **TextBase** 类，使用方法与你创建的所有 **GameObject** 均类似，详细用法参见 FAQ。
- **阳光数推荐显示在** (`SUN_COUNTER_X`, `SUN_COUNTER_Y`)。
- **剩余脑子数量与当前阶段推荐显示在右上角；关卡进度条推荐显示在屏幕下方。**
- **创建背景。**
- **创建关卡进度条**。推荐贴图编号包括：

```
1 ImageID::PROGRESS_METER_EMPTY
2 ImageID::PROGRESS_METER_FULL
3 ImageID::PROGRESS_METER_STAGE_1 // ... through STAGE_8
```

位置推荐放在屏幕下方，避免遮挡顶部卡牌栏。

- **创建推进红线**。红线位置由推进阶段和一系列缓冲区共同决定；普通僵尸只能被投放在**红线右侧**，蹦极僵尸则投放在有植物的格子上。
- **创建 5 个脑子**，每一行一个，推荐位于每行最左侧。
- **创建 45 个草坪格的植物占用记录**，`GAME_ROWS(5)` 行`GAME_COLS(9)` 列。这里不一定需要为每一格都创建可点击对象，但 **GameWorld** 应当能够知道每一格是否已经有植物。
- **按当前推进阶段生成植物防线**：你需要设计一种植物放置策略（手动配置或根据某种机制生成），让植物防线有挑战且富有游戏性。
- **生成位于顶端的僵尸卡牌按钮**：
 - 首个按钮为普通僵尸卡牌，位于 ($x = \text{ZOMBIE_CARD_FIRST_X}$, $y = \text{ZOMBIE_CARD_Y}$)。
 - 其余按钮按顺序为路障僵尸、撑杆跳僵尸、铁桶僵尸、蹦极僵尸，横向间距为 `ZOMBIE_CARD_SPACING`。
- **为所有草坪格创建僵尸投放位**。每个投放位需要记录自己的行和列；普通僵尸类只能在红线右侧且未被阻挡的格子生成，蹦极僵尸只能在已有植物的格子生成。



图 1: 关卡初始界面参考

2.2 GameWorld::Update()

你的 **GameWorld** 的 **Update()** 函数应当：

1. 遍历所有游戏对象 (**GameObject**)，并依次调用它们的 **Update()** 函数。
2. 若在上一步中有对象希望生成新的游戏对象，例如豌豆射手发射豌豆、玩家点击投放位生成僵尸，则将这些对象加入 **GameWorld** 的对象容器。
 - 如果你允许在遍历容器时直接加入对象，需要特别注意迭代器是否会失效。
 - 一种更稳妥的做法是先把新对象放入“待加入”容器，在本轮遍历结束后统一加入。
3. 检测碰撞。注意不要让同一碰撞被触发多次。可能发生的碰撞有：
 - 豌豆击中僵尸。
 - 僵尸啃咬植物。
 - 僵尸吃到某一行的脑子。
 - 撑杆跳僵尸的特殊碰撞检测可以在其 **Update()** 内完成。
 - 蹦极僵尸的抓取目标可以在其下落到目标格后由 **GameWorld** 查询。
4. 再次遍历所有游戏对象，将需要删除的对象从你的存储容器中移除。
 - 需要删除的对象应被标记为“死亡”或“HP 为零”等状态，**GameWorld** 才能找到。
 - 如果删除的是植物，不要忘记同步清空它所在草坪格的占用记录。
5. 判断是否完成当前阶段。若 5 行脑子均已被吃掉，则当前阶段结束。
 - 如果还没有完成所有阶段，则红线向右推进一格，重新开始下一阶段。下一阶段开始时，植物防线需要重新随机生成。
 - 如果当前已经是最终阶段，则此时吃掉所有脑子表示完全胜利。
6. 判断是否失败。若仍有脑子没有被吃掉，玩家没有可用阳光、场上没有可收集阳光，且场上没有任何僵尸能够继续推进，则关卡失败。
 - I, Zombie 中最便宜的可投放僵尸价格为 50，因此阳光低于 50 且没有可推进僵尸、没有可收集阳光时，玩家已经没有继续操作的空间。
 - 如果你实现了卡牌冷却，阳光足够但卡牌正在冷却时，不应立即判定失败。
7. 额外检测一遍僵尸是否在与植物发生碰撞。若某个僵尸没有和植物发生碰撞而在啃食植物，则使其正常走动。

- 这种情况的发生是由于多个僵尸在某一帧啃死了同一棵植物。先啃的僵尸并不知道植物会死亡，只能保持啃食状态，所以需要额外判断一次以使其正常移动。
8. 更新你在游戏中创建的文本显示 (**TextBase**) 的内容，以正确显示需要的信息，例如阳光数、剩余脑子数量等。
 9. 返回 `LevelStatus::ONGOING`，表示当前关卡在正常运行。

2.3 GameWorld::CleanUp()

你的 `GameWorld` 的 `CleanUp()` 函数必须清空你所使用的容器。若你保存了其他对象、草坪占用记录、脑子状态或文本对象，也需要将它们正确清除。

3 背景 (Background)

3.1 当被创建时

- 背景的贴图编号为 `ImageID::BACKGROUND`。
- 背景的位置为 ($x = \text{WINDOW_WIDTH} / 2$, $y = \text{WINDOW_HEIGHT} / 2$)。
- 背景的所在层级为 `LayerID::BACKGROUND`。
- 背景的宽度为 `WINDOW_WIDTH`，高度为 `WINDOW_HEIGHT`。
- 背景不具有动画，即，动画编号为 `AnimID::NO_ANIMATION`。

3.2 当 Update() 时

背景什么也不需要做。

3.3 当被点击时

背景什么也不需要做。

4 阳光数 (Sun Counter)

I, Zombie 中的阳光是玩家购买僵尸的资源。`GameWorld` 应当保存当前阳光总数；向日葵被吃掉时需要在场上生成可点击收集的阳光对象。

4.1 当关卡开始时

- 初始阳光推荐为 150。
- 普通僵尸价格为 50。

- 路障僵尸价格为 75。
- 撑杆跳僵尸价格为 75。
- 铁桶僵尸价格为 125。
- 蹦极僵尸价格为 125。
- 向日葵被僵尸吃掉时，掉落 150 阳光。可实现为 6 个阳光对象，每个阳光对象固定价值 25。

4.2 当 Update() 时

阳光数本身没有必须要做的事情。但 **GameWorld** 应当在每一刻更新显示阳光数量的 **TextBase**。场上的阳光对象被点击时，应通知 **GameWorld** 增加 25 阳光，并从场上消失。

4.3 当被点击时

阳光数显示不需要响应点击。

- 思考一下：僵尸卡牌需要知道玩家有没有足够阳光。这个判断应该由卡牌自己保存阳光数量完成，还是询问 **GameWorld** 完成？如果多个对象都可能改变阳光，把阳光放在什么地方最不容易出错？

5 脑子 (Brain)

脑子可以实现为 **BrainTarget**，是每一行需要被僵尸吃掉的终点。所有脑子都被吃掉后，关卡完成。

5.1 当被创建时

- 脑子的贴图编号可以为 `ImageID::BRAIN_ICON`，或你自己在 `utils.hpp` 中添加的对应贴图编号。
- 脑子的位置为每一行最左侧，推荐 `x = 35`，`y` 为对应行中心。
- 脑子的所在层级推荐为 `LayerID::UI`，或你自己定义的目标层级。
- 脑子的宽度为 32，高度为 31。
- 脑子不具有动画，即，动画编号为 `AnimID::NO_ANIMATION`。
- 脑子需要记录自己属于哪一行。

5.2 当 Update() 时

脑子什么也不需要。

5.3 当被点击时

脑子不需要响应点击。

5.4 当与僵尸碰撞时

若脑子还没有被吃掉，则脑子需要死亡，并通知 **GameWorld** 这一行已经完成。吃掉脑子的僵尸可以死亡，也可以继续向左离开屏幕；推荐让它死亡，避免重复触发同一目标。

6 推进红线与关卡进度 (Deployment Line and Progress Meter)

推进红线用于限制普通僵尸类的可投放区域，也表示当前阶段。红线左侧主要是植物防线，红线右侧是普通僵尸、路障僵尸、铁桶僵尸和撑杆跳僵尸可以投放的区域；蹦极僵尸的投放规则单独处理。

6.1 当被创建时

- 红线的贴图编号可以为 `ImageID::RED_LINE`，或你自己在 `utils.hpp` 中添加的对应贴图编号。
- 红线的位置应由当前阶段决定。推荐使用一个表示“当前植物防线推进列数”的整数，例如 `m_deploymentStartCol`，再额外保留一列缓冲区。普通僵尸的最左可投放列可以为 `m_deploymentStartCol + ZOMBIE_DEPLOYMENT_BUFFER_COLS`。
- 游戏共有 5 个阶段。初始阶段中：

```
1 m_deploymentStartCol = INITIAL_ZOMBIE_DEPLOYMENT_START_COL;
```

每完成一个阶段，`m_deploymentStartCol` 增加 1。

- 最终阶段由 `FINAL_ZOMBIE_DEPLOYMENT_START_COL` 决定。它等于：

```
1 INITIAL_ZOMBIE_DEPLOYMENT_START_COL + TOTAL_ROUNDS - 1
```

- 进度条的贴图编号可以为：

```
1 ImageID::PROGRESS_METER_EMPTY
2 ImageID::PROGRESS_METER_FULL
3 ImageID::PROGRESS_METER_STAGE_1 // ... through STAGE_8
```

6.2 当阶段完成时

若所有脑子均被吃掉，**GameWorld** 应当检查当前是否为最终阶段。若不是最终阶段，红线向右移动一格，并重新开始下一阶段；若已经是最终阶段，则游戏完全胜利。

6.3 当新阶段开始时

- 清空上一阶段残留的植物、僵尸、豌豆、阳光等对象。
- 将阳光数重置为初始阳光。
- 重置每一行脑子。
- 根据新的红线位置，更新普通僵尸的可投放范围。
- 按当前阶段随机生成植物。
- 更新进度 UI 与文本。

7 僵尸投放位 (Zombie Deployment Lane)

僵尸投放位可以实现为 **Zombie Deployment Lane**，是位于草坪格中的“隐形的兔子”。点击它可以把手上的僵尸放到对应的行和列。普通僵尸、路障僵尸、铁桶僵尸和撑杆跳僵尸只能放在红线右侧；蹦极僵尸用于偷取植物，因此只能放在已有植物的格子上。

7.1 当被创建时

- 僵尸投放位的贴图编号为 `ImageID::NONE`，没有贴图。
- 僵尸投放位的位置并不固定，而由创建它的代码决定。（因此，它的构造函数中必须包含此部分。）
- 僵尸投放位的所在层级为 `LayerID::UI`。
- 僵尸投放位的宽度推荐为 `LAWN_GRID_WIDTH`，高度推荐为 `LAWN_GRID_HEIGHT - 10`。
- 僵尸投放位不具有动画，即，动画编号为 `AnimID::NO_ANIMATION`。
- 僵尸投放位需要记录自己对应哪一行、哪一列。

7.2 当 Update() 时

僵尸投放位什么也不需要做。

7.3 当被点击时

若玩家正拿着一个还未投放的僵尸卡牌，则尝试“投放”它，即在该投放位对应的草坪格生成一个僵尸。

- 如果该格附近已有僵尸，点击应当无效，以避免多个僵尸完全重叠。
- 如果手上拿的是普通僵尸类卡牌，而该格不在红线右侧，点击应当无效。
- 如果手上拿的是蹦极僵尸卡牌，而该格没有植物，点击应当无效。

- 投放成功后，应当扣除阳光，清空玩家手上的选择，并让对应卡牌进入冷却。
- 思考一下：既然在点击时才需要生成僵尸，那么玩家拿在手里的就并不是僵尸，当然也不是卡牌按钮本身。这个东西究竟是什么？提示：另一种隐形的兔子！

8 普通僵尸卡牌 (Regular Zombie Card)

8.1 当被创建时

- 普通僵尸卡牌的贴图编号为 `ImageID::ZOMBIE_CARD_REGULAR`，或你自己在 `utils.hpp` 中添加的对应贴图编号。
- 普通僵尸卡牌位于第一个僵尸卡牌位置 (`x = ZOMBIE_CARD_FIRST_X`, `y = ZOMBIE_CARD_Y`)。
- 普通僵尸卡牌的所在层级为 `LayerID::UI`。
- 普通僵尸卡牌的宽度为 50，高度为 70。
- 普通僵尸卡牌不具有动画。
- 普通僵尸卡牌的价格是 50 阳光。
- 普通僵尸卡牌的冷却时间推荐为 120 个 tick (4 秒)。

8.2 当 Update() 时

普通僵尸卡牌可以在 `Update()` 中减少冷却时间。

8.3 当被点击时

1. 如果玩家正拿着一个还未投放的僵尸，则点击无效，或取消当前选择后重新选择，具体由你的交互设计决定。
 2. 如果普通僵尸卡牌正处于冷却状态，则点击无效。（如果你实现了冷却遮罩，则遮罩会遮挡住点击）
 3. 询问 `GameWorld` 是否有 50 阳光。如果有，则让玩家在下次点击投放位时投放普通僵尸。卡牌冷却在投放成功后开始。
- 注意，点击卡牌时不应立刻花费阳光，因为只有点击投放位后才能知道僵尸会出现在哪一格。
 - 如果投放失败，不应扣除阳光，也不应开始冷却；只有投放成功后，卡牌才会进入冷却。
 - 思考一下：每种僵尸都会有一个其对应的卡牌，不同僵尸的卡牌贴图、冷却时间、价格会有区别，除此以外还有区别吗？既然如此，不同僵尸的卡牌究竟是属于不同的类 (**Class**) 还是不同的实例 (**Instance**)？如何让僵尸和其对应的卡牌联系在一起？提示：模板类 (**Template**)。

9 冷却遮罩 (Cooldown Mask)

9.1 当被创建时

- 冷却遮罩的贴图编号为ImageID::COOLDOWN_MASK。
- 冷却遮罩的位置由创建它的代码决定。
- 冷却遮罩的所在层级为LayerID::COOLDOWN_MASK。(会遮挡卡牌)
- 冷却遮罩的宽度为 50，高度为 70。
- 冷却遮罩不具有动画。
- 冷却遮罩需要在对应的僵尸卡牌冷却后消失。

9.2 当 Update() 时

冷却遮罩需要在对应的僵尸卡牌冷却后消失。

9.3 当被点击时

冷却遮罩什么也不需要做。

10 向日葵 (Sunflower)

10.1 当被创建时

- 向日葵的贴图编号为ImageID::SUNFLOWER。
- 向日葵的位置并不固定，而由创建它的代码决定。(因此，它的构造函数中必须包含此部分。)
- 向日葵的所在层级为LayerID::PLANTS。
- 向日葵的宽度为 60，高度为 80。
- 向日葵具有AnimID::IDLE 动画。
- 向日葵具有 340 点HP。

10.2 当 Update() 时

向日葵需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回。I, Zombie 中向日葵不需要主动产生阳光；它的作用是在被僵尸吃掉时为玩家提供阳光。

10.3 当被点击时

如果玩家手中拿着蹦极僵尸卡牌，点击向日葵所在格可以尝试投放蹦极僵尸；否则向日葵不需要响应点击。

10.4 当被僵尸吃掉时

向日葵需要死亡，并通知 **GameWorld** 掉落 150 点阳光。推荐掉落 6 个可点击阳光对象，每个价值 25。

11 豌豆射手 (Peashooter)

11.1 当被创建时

- 豌豆射手的贴图编号为ImageID::PEASHOOTER。
- 豌豆射手的位置由创建它的代码决定。
- 豌豆射手的所在层级为LayerID::PLANTS。
- 豌豆射手的宽度为 60，高度为 80。
- 豌豆射手具有AnimID::IDLE 动画。
- 豌豆射手具有 340 点HP。
- 豌豆射手每 32 tick 左右发射一颗豌豆。

11.2 当 Update() 时

1. 豌豆射手需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回，无视下述步骤。
2. 如果豌豆射手的攻击能力正在冷却，则降低 1 冷却时间，它的Update() 返回。
3. 如果豌豆射手可以攻击，它将询问 **GameWorld**，自己的一行上，自己的右边是否存在僵尸。若存在，则在豌豆射手自身坐标的右方 30 像素、上方 20 像素位置生成一颗豌豆，然后进入约 32 tick 的冷却时间。

11.3 当被点击时

如果玩家手中拿着蹦极僵尸卡牌，点击豌豆射手所在格可以尝试投放蹦极僵尸；否则豌豆射手不需要响应点击。

12 双发射手 (Repeater)

双发射手是后续阶段会出现的植物。它和豌豆射手有相同的基本碰撞、HP 和发射检测方式，但一次攻击会连续发射两颗豌豆。

12.1 当被创建时

- 双发射手的贴图编号为ImageID::REPEATER。
- 双发射手的位置由创建它的代码决定。
- 双发射手的所在层级为LayerID::PLANTS。
- 双发射手的宽度为 60，高度为 80。
- 双发射手具有AnimID::IDLE 动画。
- 双发射手具有 340 点HP。

12.2 当 Update() 时

1. 双发射手需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回。
2. 如果双发射手的第二发豌豆正在等待，则减少等待时间；等待时间归零时发射第二颗豌豆。
3. 如果双发射手可以攻击，它将询问 **GameWorld**，自己的一行上，自己的右边是否存在僵尸。若存在，则立刻发射第一颗豌豆，进入攻击冷却，并设置一个较短的第二发等待时间。

12.3 当被点击时

如果玩家手中拿着蹦极僵尸卡牌，点击双发射手所在格可以尝试投放蹦极僵尸；否则双发射手不需要响应点击。

13 豌豆 (Pea)

13.1 当被创建时

- 豌豆的贴图编号为ImageID::PEA。
- 豌豆的位置由创建它的代码决定。
- 豌豆的所在层级为LayerID::PROJECTILES。
- 豌豆的宽度为 28，高度为 28。
- 豌豆不具有动画。

13.2 当 Update() 时

1. 豌豆需要首先检查自己是否已经死亡。若已经死亡, 它将等待 **GameWorld** 清理。它的 `Update()` 应当立刻返回, 无视下述步骤。
2. 豌豆将向右移动 8 像素。
3. 如果豌豆飞出了屏幕右边界 ($x \geq \text{WINDOW_WIDTH}$), 豌豆需要死亡。

13.3 当被点击时

豌豆什么也不需要做。

13.4 当与僵尸碰撞时

豌豆将会对与它碰撞的第一个僵尸造成 24 点伤害, 然后死亡。

14 坚果墙 (Wall-nut)

14.1 当被创建时

- 坚果墙的贴图编号为 `ImageID::WALLNUT`。
- 坚果墙的位置由创建它的代码决定。
- 坚果墙的所在层级为 `LayerID::PLANTS`。
- 坚果墙的宽度为 60, 高度为 80。
- 坚果墙具有 `AnimID::IDLE` 动画。
- 坚果墙具有 3600 点HP。

14.2 当 Update() 时

1. 坚果墙需要首先检查自己是否已经死亡。若已经死亡, 它将等待 **GameWorld** 清理。它的 `Update()` 应当立刻返回, 无视下述步骤。
2. 如果坚果墙的HP 小于总HP 的 1/3, 将它的贴图更换为“受损的坚果墙”:

```
1 ImageID::WALLNUT_CRACKED
```

14.3 当被点击时

如果玩家手中拿着蹦极僵尸卡牌, 点击坚果墙所在格可以尝试投放蹦极僵尸; 否则坚果墙不需要响应点击。

15 普通僵尸 (Regular Zombie)

15.1 当被创建时

- 普通僵尸的贴图编号为 `ImageID::REGULAR_ZOMBIE`。
- 普通僵尸的位置由创建它的代码决定, 推荐为投放格中心, 即 $x = \text{ColToX}(\text{col}), y = \text{RowToY}(\text{row})$ 。
- 普通僵尸的所在层级为 `LayerID::ZOMBIES`。
- 普通僵尸的宽度为 20, 高度为 80, 如图。这样设定是为了防止它同时与两棵植物碰撞。
- 普通僵尸初始具有 `AnimID::WALK` 动画。
- 普通僵尸具有 260 点HP。
- 普通僵尸的价格是 50 阳光。

15.2 当 `Update()` 时:

1. 普通僵尸需要首先检查自己是否已经死亡。若已经死亡, 它将等待 `GameWorld` 清理。它的 `Update()` 应当立刻返回, 无视下述步骤。
2. 若普通僵尸当前正在行走, 它将向左移动 1 像素。若普通僵尸当前正在啃食植物, 它将不会移动。

15.3 当被点击时

普通僵尸什么也不需要做。

15.4 当发生碰撞时



图 2: 碰撞判定与普通僵尸碰撞体参考

- 如果与一颗豌豆发生碰撞，普通僵尸将会受到 24 点伤害，并让那颗豌豆死亡。
- 如果与任何植物发生碰撞，普通僵尸将会：
 - 如果当前正在行走，则进入啃食状态，并播放AnimID::EAT动画。
 - 对那棵植物造成 4 点伤害。（这是每帧 4 点的持续伤害，即每秒约 120 点）
- 如果与本行脑子发生碰撞，普通僵尸将吃掉脑子，并通知GameWorld 这一行完成。

16 路障僵尸 (Conehead Zombie)

16.1 当被创建时

- 路障僵尸的贴图编号为 ImageID::CONEHEAD_ZOMBIE。
- 路障僵尸的位置由创建它的代码决定。
- 路障僵尸的所在层级为LayerID::ZOMBIES。
- 路障僵尸的宽度为 20，高度为 80。
- 路障僵尸初始具有AnimID::WALK 动画。
- 路障僵尸具有 560 点HP。
- 路障僵尸的价格是 75 阳光。

16.2 当 Update() 时:

1. 路障僵尸需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回。
2. 若路障僵尸当前正在行走，它将向左移动 1 像素。若路障僵尸当前正在啃食植物，它将不会移动。
3. 当路障僵尸HP 小于等于 200 点时，可以将它的贴图更换为普通僵尸的贴图。

16.3 当发生碰撞时

路障僵尸的碰撞行为与普通僵尸相同。

17 铁桶僵尸 (Buckethead Zombie)

17.1 当被创建时

- 铁桶僵尸的贴图编号为ImageID::BUCKET_HEAD_ZOMBIE。

- 铁桶僵尸的位置由创建它的代码决定。
- 铁桶僵尸的所在层级为LayerID::ZOMBIES。
- 铁桶僵尸的宽度为 20，高度为 80。
- 铁桶僵尸初始具有AnimID::WALK 动画。
- 铁桶僵尸具有 1450 点HP。
- 铁桶僵尸的价格是 125 阳光。

17.2 当 Update() 时:

1. 铁桶僵尸需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回，无视下述步骤。
2. 若铁桶僵尸当前正在行走，它将向左移动 1 像素。若铁桶僵尸当前正在啃食植物，它将不会移动。
3. 若铁桶僵尸的HP 小于等于 200 点，它将失去铁桶。将它的贴图更换为普通僵尸的贴图。

17.3 当发生碰撞时

铁桶僵尸的碰撞行为与普通僵尸相同。

18 撑杆跳僵尸 (Pole Vaulting Zombie)

18.1 当被创建时

- 撑杆跳僵尸的贴图编号为ImageID::POLE_VAULTING_ZOMBIE。
- 撑杆跳僵尸的位置由创建它的代码决定。
- 撑杆跳僵尸的所在层级为LayerID::ZOMBIES。
- 撑杆跳僵尸的宽度为 20，高度为 80。
- 撑杆跳僵尸初始具有AnimID::RUN 动画。
- 撑杆跳僵尸具有 420 点HP。
- 撑杆跳僵尸的价格是 75 阳光。

18.2 当 Update() 时:

1. 撑杆跳僵尸需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。它的Update() 应当立刻返回，无视下述步骤。

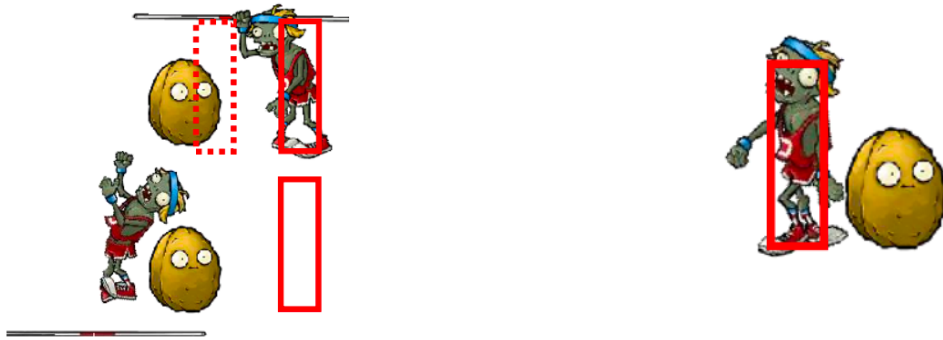


图 3: 撑杆跳僵尸跳跃判定参考

2. 若撑杆跳僵尸当前正在奔跑，它将检测左方 40 像素位置是否有可以跳过的植物：

- a) 它会在这一 tick，暂时将自己向左移动 40 像素。
- b) 询问 **GameWorld**，它是否在与一棵植物碰撞。如果是，它将：
 - 停止移动（为了和动画保持一致），播放 `AnimID::JUMP` 动画。在 42 帧后转为行走（`AnimID::WALK`）并向左“瞬移”150 像素（还是为了和动画保持一致）。
- c) 无论它是否在这一帧触发了 b)，不要忘了再向右移回 40 像素以撤回你在 a) 中的临时修改。

3. 若撑杆跳僵尸当前正在奔跑，它将向左移动 2 像素。若撑杆跳僵尸当前正在行走，它将向左移动 1 像素。若撑杆跳僵尸当前正在啃食植物或跳过植物，它将不会移动。

18.3 当发生碰撞时

- 如果与一颗豌豆发生碰撞，撑杆跳僵尸将会受到 24 点伤害，并让那颗豌豆死亡。
- 如果与任何植物发生碰撞，撑杆跳僵尸将会：
 - 如果当前正在行走，则进入啃食状态，并播放 `AnimID::EAT` 动画。
 - 对那棵植物造成 4 点伤害。
- 如果与本行脑子发生碰撞，撑杆跳僵尸将吃掉脑子，并通知 **GameWorld** 这一行完成。

19 蹦极僵尸 (Bungee Zombie)

蹦极僵尸不是从右向左推进的僵尸，而是用于偷取某一格植物的特殊僵尸。玩家选择蹦极僵尸卡牌后，应点击一个已有植物的草坪格；若该格没有植物，投放失败。

19.1 当被创建时

- 蹦极僵尸的贴图编号为ImageID::BUNGEE_ZOMBIE。
- 蹦极僵尸的位置由创建它的代码决定，推荐以目标格中心作为落点，并从落点上方一定距离开始下落。
- 蹦极僵尸的所在层级为LayerID::ZOMBIES。
- 蹦极僵尸的宽度为 20，高度为 80。
- 蹦极僵尸具有 450 点HP。
- 蹦极僵尸的价格是 125 阳光。

19.2 当 Update() 时

1. 蹦极僵尸需要首先检查自己是否已经死亡。若已经死亡，它将等待 **GameWorld** 清理。
2. 若蹦极僵尸处于下落阶段，它将向目标格移动；到达目标格后切换到抓取贴图或抓取阶段。
3. 若蹦极僵尸处于抓取阶段，它会等待数个 tick，并在指定时机移除目标格中的植物。
4. 抓取阶段结束后，蹦极僵尸进入撤离阶段，向上离开屏幕或目标区域，随后死亡并等待 **GameWorld** 清理。

19.3 当发生碰撞时

蹦极僵尸不会像普通僵尸那样啃食植物。它的主要行为是在抓取阶段从 **GameWorld** 查询自己所在格的植物，并对其造成足够高的伤害，使植物死亡。

20 附录 - 碰撞处理

每个对象的碰撞体均为中心在 (x, y)，横向宽度为width, 纵向高度为height 的长方形。若两对象的碰撞体长方形发生重叠，则认为两对象发生碰撞。提示：碰撞关系似乎看起来有些复杂：有可能左右或上下交叉，有可能一个包围另一个，但实际上有非常简单的判断方法：考虑两个对象的中心点，是否碰撞是否跟他们中心点之间的 x 或 y 距离有关？